Size Change Termination Condition for Injective Relations

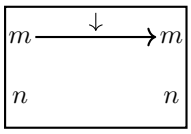
Stefano Berardi

April, 2025

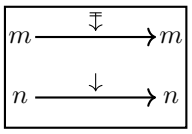
1 Introduction

Size change termination (SCT), originally introduced in [LJBA01], is a method used to prove the termination of recursive functions. The main idea behind SCT lies in Fermat's proof principle of infinite descent: to proof a proposition P, assume P is false and reach a contradiction by constructing an infinite descending chain in a well-order. This principle can be applied in the context of recursive functions as follows: if the variables of a function belong to a well-order, we can assume the function execution does not terminates, and derive a contradiction by forming an infinite descending chain with the values of the variables. These infinite descending chains are formed by analyzing the relationships between the variables of the function in each recursive call. Each relationship is captured by a structure known as the *size change graph*, which provides an abstract way to define how the variables change during program execution. To understand the general idea behind this method, lets consider the following example of the Ackermann function, taken from [Lee09]:

```
a(m,n) = if m=0 then n+1 else
         if n=0 then 1a(m-1,1) else
         2a(m-1,3a(m,n-1))
```



G_1, G_2



G_3

Figure 1: Definition of the Ackermann function `a` and size change graphs G_1, G_2 and G_3

For each recursive call, labeled with 1,2 and 3, we can respectively define the size change graphs G_1, G_2 and G_3, where the edges $x \xrightarrow{\gamma} y$ represent the relationship between the values of the variables initially passed to the function, and the values after each recursive call. When $\gamma = \downarrow$, we call it a *progressing* edge and it specifies that the value of x is strictly less than the value of y. When $\gamma = \mathbin{\downarrow\!\!\!=}$, we call it a *stationary* edge, and it specifies that the value of x is less than or equal to the value of y. Using these graphs we can then analyze the function's execution: assume a hypothetical infinite execution of the program. This will induce an infinite sequence of recursive calls $c = c_1c_2c_3...$ where every $c_i \in \{1, 2, 3\}$, which also induces an infinite sequence of size change graphs $G_{c_1}G_{c_2}G_{c_3}...$ It is then possible to compose these graphs, to form infinite paths using the relations between each one of the variables. For example, suppose we get an infinite execution $(13)^\omega$, i.e. the recursive call 1 and 3 are used infinitely one after the other. This induces the sequence of graphs $(G_1G_3)^\omega$ and the infinite path $m \xrightarrow{\downarrow} m \xrightarrow{\mathbin{\downarrow\!\!\!=}} m \xrightarrow{\downarrow} m...$ over the variable m, which progresses infinitely often and implies that this infinite execution is impossible. Given this intuition, we can define the SCT property as follows:

Definition 1.1. A program satisfies SCT if and only if for any possible infinite sequence of recursive calls $c = c_1c_2c_3...$, the induced composition of infinite size change graphs $G_{c_1}G_{c_2}G_{c_3}...$ has an infinite path that infinitely progresses, i.e. the edge .

This notion of SCT is closely related to the *global trace condition* (GTC), which is used to define the validity of *circular proofs*. More precisely, is has been known in the folk-lore that SCT is equivalent to GTC, and also proven for particular circular proof systems like in [cite Mirai if possible]. To explain the intuition behind

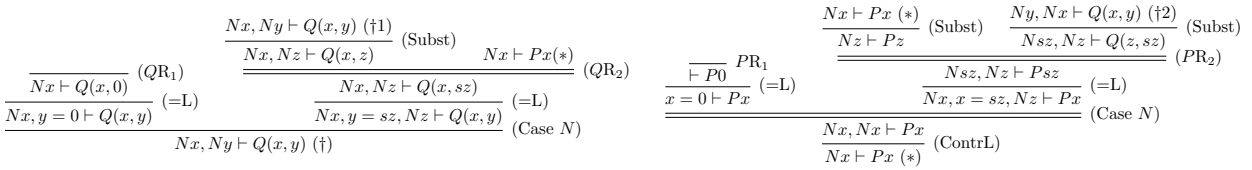


Figure 2: Example of a circular proof in CLKID$^\omega$. The rightmost branch ($*$) on the proof on the left continues in the root of the proof on the right.

this equivalence, consider the following example of a circular proof in the CLKID$^\omega$ circular proof system, taken from [Bro06]:

We are going to change the narrative of the introduction:

1. We define and explain the generality of the size change termination condition (def + example).
2. We define and explain the generality of the global trace condion (def + example)
3. We remark that both of them are equivalent and can be seen as one definition given an abstract framework using only relations and the underlying structure of the program/proof tree
4. Then we introduce PDC with injectivity
5. We argue why circular proofs are in general injective
6. We explain what are the advantages of using PDC + Injectivity over the abstract version of STC/GTC

References

[Bro06] James Brotherston. Sequent calculus proof systems for inductive definitions. 2006.

[Lee09] Chin Soon Lee. Ranking functions for size-change termination. *ACM Trans. Program. Lang. Syst.*, 31(3), April 2009.

[LJBA01] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The size-change principle for program termination. In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '01, page 81–92, New York, NY, USA, 2001. Association for Computing Machinery.